

AI-BASED RESUME SCREENING AND AI-ASSISTED INTERVIEW EVALUATION FRAMEWORK WITH PROCTORING SYSTEM

Author Name
Affiliation
author@email.com

Author Name
Affiliation
author@email.com

Author Name
Affiliation
author@email.com

Abstract - Automated resume screening typically relies on keyword or lexical matching and produces a single static score, with no mechanism to revise that judgment when later evidence—such as an interview—contradicts it. This paper presents an integrated, locally-deployable AI recruitment pipeline combining resume-job description matching, adaptive AI interviewing, structured interview evaluation, and integrity/proctoring analysis, and proposes an evaluation framework for testing whether an interview stage corrects ranking errors introduced during resume-only screening. We define two controlled audit conditions—an "over-seller" case and a "hidden gem" case—with rank-shift and recovery-rate metrics to quantify correction behavior, alongside baseline comparisons (TF-IDF/BM25, Sentence-BERT, zero-shot LLM), an ablation over pipeline components, adversarial robustness testing, and a local-versus-hosted model comparison on quality, latency, and resource usage. [RESULT NOT YET AVAILABLE: quantitative outcomes are pending experiment execution.] The contribution is framed as (i) an integrated pipeline architecture, (ii) a reproducible rank-consistency audit methodology, and (iii) an experimental design testing—rather than assuming—screening improvement; fairness/bias auditing is explicitly out of scope pending demographic evaluation data.

Keywords – Artificial Intelligence, Recruitment Automation, Large Language Models, Data Privacy, Interview Evaluation

INTRODUCTION

Large-scale recruitment pipelines depend on an initial screening stage to reduce a large applicant pool to a shortlist suitable for human review. Traditional applicant tracking systems (ATS) perform this screening primarily through keyword or rule-based matching between resume text and job description requirements [REQUIRED CITATION]. More recent systems incorporate statistical

and neural language representations—ranging from term-frequency and lexical-overlap methods to transformer-based embedding similarity—to capture semantic relevance beyond exact keyword overlap [REQUIRED CITATION]. In parallel, the emergence of large language models (LLMs) has enabled automated generation and evaluation of open-ended interview responses, extending automation from document screening into structured candidate assessment [REQUIRED CITATION].

Despite this progress, existing systems predominantly treat resume screening and interview evaluation as independent, sequential filters rather than as a single pipeline in which later evidence can revise an earlier judgment. A resume-based score is typically computed once and used to gate access to subsequent stages, but once a candidate proceeds to interview, prior work offers little methodological basis for asking whether the interview stage actually *corrects* errors present in the initial resume-based ranking—for example, a resume that overstates a candidate's competence, or one that understates it. Existing evaluations of recruitment AI systems also tend to report accuracy or agreement for a single component in isolation (screening or interview scoring), rather than auditing the consistency of the ranking across pipeline stages, and rarely report robustness to adversarial resume content such as keyword inflation or prompt injection within the same evaluation framework [REQUIRED CITATION]. This leaves an open question as to whether integrating adaptive interviewing with resume screening yields a measurably more consistent candidate ranking than resume screening alone, or whether it simply adds computational and operational cost without a demonstrable correction effect.

This paper addresses that gap by proposing an integrated, locally-deployable recruitment pipeline that combines

Commented [A1]: Do not complete the Author name section until you submit the final camera-ready copy.

Important notice about copyright:

During the final paper submission process, you must complete the electronic copyright form. Your paper cannot be published in the Proceedings if you do not complete this step.

There are specific copyright notices for Government agencies. Please contact the Proceedings Co-chairs for the correct statement to include in your paper.

Delete this comment before submitting your paper for the first review.

resume-job description matching, adaptive AI-driven interviewing, structured interview-answer evaluation, and integrity/proctoring analysis, together with a rank-consistency audit methodology designed to measure—rather than assume—whether the interview stage corrects resume-only screening errors under controlled conditions.

At the time of writing, the experimental results supporting this methodology have not yet been executed; consequently, this paper reports the pipeline design, evaluation framework, and experimental protocol rather than final quantitative outcomes [RESULT NOT YET AVAILABLE]. The contributions of this work are therefore methodological and architectural rather than result-driven, and are limited to the following:

An integrated, provider-agnostic recruitment pipeline architecture supporting both hosted and locally-deployed LLMs, enabling controlled comparison of evaluation quality, latency, and resource consumption across deployment modes.

A rank-consistency audit methodology, comprising an "over-seller" and a "hidden gem" controlled condition with defined rank-shift and recovery-rate metrics, intended to test whether adaptive interviewing corrects resume-only ranking errors.

A baseline and ablation framework comparing lexical, embedding-based, and zero-shot LLM screening methods against the proposed pipeline, together with an adversarial robustness protocol for resume-based prompt injection and manipulation.

An explicit delimitation of scope, in which fairness/bias auditing is identified as future work pending appropriate demographic evaluation data, rather than claimed as an established property of the system.

The rest of this paper is organized as follows. Section II reviews related work in automated resume screening, LLM-based interview evaluation, and adversarial robustness of language-model systems. Section III formulates the recruitment ranking problem addressed in this work. Section IV describes the proposed pipeline architecture. Section V details the experimental methodology, including baselines, metrics, and the rank-consistency audit design. Section VI discusses threats to validity and ethical considerations. Section VII concludes and outlines future work.

LITERATURE SURVEY

Automated recruitment technology has evolved through several distinct methodological generations, moving progressively from surface-level text matching toward context-sensitive, model-driven candidate evaluation. Early systems were designed primarily to manage applicant volume rather than to assess candidate quality with any semantic depth, and this design goal has

continued to shape the field even as the underlying language technology has changed substantially. The following review traces this evolution across four related but distinct lines of work — rule-based applicant tracking, semantic and embedding-based matching, transformer- and LLM-based recruitment systems, and the explainability, fairness, and security concerns that these more capable systems have introduced — before identifying where the present work situates itself relative to this trajectory.

The earliest generation of applicant tracking systems (ATS) treated resume screening chiefly as a document-retrieval and scoring problem, ranking candidates by the presence or frequency of employer-specified keywords and criteria [1]. Tiwari et al. [1] describe an applicant tracking and scoring architecture representative of this generation, in which resumes are parsed into structured fields and compared against job requirements through largely deterministic matching logic. This approach offered clear advantages in processing speed and auditability, since the scoring criteria were explicit and reproducible, but it provided no mechanism for recognizing that two differently worded skill descriptions might refer to the same underlying competency—a limitation that motivated the subsequent shift toward semantic representations.

Subsequent work sought to close this semantic gap by representing resumes and job descriptions in a shared vector space rather than as literal token sequences. Pudasaini et al. [3] combined word-embedding representations with a Gale-Shapley stable-matching procedure, computing cosine similarity between embedded resume and job-description text and then resolving the resulting bilateral preferences into a stable assignment between candidates and positions. This formulation reframed screening as an optimization problem over mutual preference rather than a one-directional ranking task, which is a meaningfully different objective from the ranking-only formulation adopted by most later systems, including the present work. In contrast, approaches built on named-entity recognition and keyword-augmented embeddings pursued interpretability more directly by explicitly extracting skill and requirement entities before computing similarity, trading some of the flexibility of purely distributional representations for scores that could be traced back to specific extracted terms. Across this line of work, a common limitation is that embedding similarity, however computed, is optimized to reflect textual relatedness rather than an externally validated notion of job-fit, so improvements in similarity metrics do not by themselves establish improvements in hiring-relevant accuracy.

Contextual transformer models extended this semantic-matching paradigm by conditioning a candidate's representation on the surrounding resume section and, in

some formulations, on the paired job description jointly rather than encoding each document independently. Li et al. [2] demonstrated this shift concretely in a large-scale resume-classification study, in which a context-aware transformer architecture using section encoding and multi-head attention decoding was evaluated against a corpus of 6,492 clinical-research-coordinator resumes annotated through triple-blind expert review; their models achieved 73.3% accuracy on competence-level classification and 79.2% accuracy on the paired resume–job-description matching task, with an inter-annotator Kappa of 61% establishing that even expert human judgment does not converge perfectly on adjacent competence levels. This result is significant for the present work in two respects: it shows that contextual encoding measurably outperforms non-contextual similarity on a rigorously annotated dataset, and it establishes that human-agreement ceilings, not only model capacity, bound how much matching accuracy can improve — a consideration directly relevant to the human-agreement experiments proposed in Section IV.

The most recent generation of systems replaces fixed embedding comparison with generative, agentic LLM-based evaluation. Gan et al. [5] introduced an LLM-agent framework that summarizes and grades resumes at scale, reporting screening roughly eleven times faster than manual review and an F1 score of 87.7% on resume-sentence classification after fine-tuning, with LLM agents additionally participating in downstream offer-stage decision-making. Extending this direction toward interpretability, subsequent multi-agent designs decompose the screening task across specialized extractor, evaluator, and summarizer agents, in some cases incorporating retrieval-augmented generation to ground scores in external role-specific criteria rather than relying solely on parametric knowledge [6]. Compared with the single-pass embedding-similarity methods discussed above, these LLM-agent architectures produce natural-language justifications alongside numeric scores, which is a qualitatively different output than a similarity scalar; however, this capability introduces a corresponding evaluation burden, since a generated justification's fluency does not guarantee that it accurately reflects the reasoning that produced the associated score.

The growing capability and deployment scale of these systems has drawn increased scrutiny of their fairness and robustness properties. Fabris et al. [7] provide a multidisciplinary survey of fairness and bias across the algorithmic hiring pipeline, cataloguing domain-specific bias sources, mitigation techniques, and the legal frameworks — including EU and US regulatory instruments — that govern their deployment, while explicitly noting that whether algorithmic hiring is less biased than the human-driven alternatives it replaces remains, in their assessment, an open and evidence-

dependent question rather than a settled conclusion. Separately, the generative and instruction-following nature of LLM-based screening has introduced a security dimension absent from earlier keyword- and embedding-based systems: Baxi et al. [8] show experimentally that self-promotional prompt-injection text embedded in a résumé can reliably improve an applicant's LLM-assigned ranking when few candidates employ the tactic and résumé quality is otherwise homogeneous, but that this advantage collapses as injection becomes widespread, and can in heterogeneous-quality settings occasionally invert the ranking of otherwise weaker and stronger candidates — a finding with direct fairness implications once manipulation is unevenly distributed across an applicant pool.

Across these four lines of work, resume screening and interview-stage evaluation continue to be treated predominantly as independent problems: fairness and robustness studies examine screening or generation in isolation, and LLM-agent frameworks that incorporate multiple pipeline stages report efficiency and classification accuracy rather than auditing whether a later stage corrects errors introduced by an earlier one. None of the reviewed work formulates a controlled test of whether an interview stage measurably revises a resume-only ranking that later evidence contradicts, and the robustness finding in [8] is reported for screening in isolation rather than for a pipeline in which a subsequent interview stage might counteract an injection-inflated resume score. The present work addresses this gap directly by treating resume screening and adaptive interviewing as stages of a single auditable ranking process and by defining rank-shift and recovery-rate metrics that test, under controlled over-seller and hidden-gem conditions, whether the interview stage corrects — rather than merely supplements — the resume-based screening judgment reviewed in this section.

PROPOSED SYSTEM

A. System Overview

Fig. 1 illustrates the architecture of the proposed recruitment pipeline, which was structured as a sequence of loosely coupled services rather than a monolithic model. Given a candidate resume and a target job description, the system performed (i) resume parsing and structured field extraction, (ii) semantic resume–job description matching to produce an initial screening score, (iii) adaptive interview generation and response evaluation for shortlisted candidates, (iv) integrity/proctoring signal aggregation during the interview session, and (v) score fusion to produce a final ranked candidate report for recruiter review. Each stage was implemented as an independently testable service so that individual components — e.g., the matching function or the interview scorer — could be substituted or ablated without modifying the surrounding pipeline. [REQUIRED FIGURE: Fig. 1 — block diagram showing Resume → Parser → Matcher →

Screening Score $\rightarrow$ (gate) $\rightarrow$ Adaptive Interview $\rightarrow$
Interview Score $\rightarrow$ Integrity Analysis $\rightarrow$ Score Fusion $\rightarrow$
Ranked Report $\rightarrow$ Recruiter.]

B. Problem Formulation

Let R denote a candidate resume and J a job description, both represented as raw text. Let $f_\theta(\cdot)$ denote the resume-job matching function, parameterized by θ (e.g., a fixed lexical statistic, a frozen embedding model, or an LLM prompt configuration, depending on the method under evaluation). The screening score was defined as

$$S_{\text{resume}} = f_\theta(R, J) \in [0, 1].$$

Candidates were shortlisted using a threshold τ_{ats} , such that a candidate proceeded to interview if $S_{\text{resume}} \geq \tau_{\text{ats}}$; otherwise the candidate was rejected prior to any interview stage, consistent with the two-stage structure of the underlying system. For a shortlisted candidate, let $I = i_1, \dots, i_k$ denote the sequence of k interview turns, where each i_t is a (question, answer) pair. The interview evaluation function $g_\phi(\cdot)$ produced

$$S_{\text{interview}} = g_\phi(I, J, R) \in [0, 1].$$

Let E denote the set of integrity/proctoring events observed during the interview session, and let $c(E) \in [0, 100]$ denote an integrity score derived from E (Section III-F). The final candidate score was computed as

$$S_{\text{final}} = h(S_{\text{resume}}, S_{\text{interview}}, c(E)),$$

where $h(\cdot)$ denotes the fusion function. **[REQUIRED SPECIFICATION: the exact functional form of $h(\cdot)$ e.g., a fixed weighted sum $S_{\text{final}} = w_1 S_{\text{resume}} + w_2 S_{\text{interview}}$ with $c(E)$ used only as a gating/flagging signal rather than a multiplicative term — must be taken from the actual implementation and reported here verbatim, together with the values of w_1, w_2 if fixed weights are used. This paper does not assume a weighting scheme that has not been confirmed against the system's source.]** Candidates were ranked by S_{final} to produce the recruiter-facing report. S_{final} is treated throughout this paper as a decision-support signal rather than an autonomous accept/reject decision; the recruiter retains final authority over disposition.

C. Stage 1: Resume Parsing and Field Extraction

Raw resume text (plain text or PDF-extracted text) was parsed into structured fields — including, at minimum, skills, prior roles, and years of experience — prior to matching. **[REQUIRED SPECIFICATION: parsing method — rule-based/regex extraction, a named-entity recognition model, or an LLM-based extraction prompt — and, if an LLM is used, the exact prompt template and output schema, since this determines what information the matching stage in Section III-D can access.]** Parsing failures (e.g., unparseable PDFs, empty sections) were logged and excluded from the evaluation set rather than silently imputed, to avoid conflating parser error with matching error in downstream metrics.

D. Stage 2: Resume–Job Description Matching

For each candidate–job pair, S_{resume} was computed under one of five matching configurations, corresponding to the proposed method and four baselines (Section III-G): TF-IDF cosine similarity, BM25, Sentence-BERT embedding cosine similarity, zero-shot LLM scoring, and the proposed method. For the proposed method, **[REQUIRED SPECIFICATION: state precisely whether f_θ is an LLM-prompted rubric score, a fine-tuned classifier, or a hybrid of lexical and embedding signals, and give the exact prompt or feature set].** All matching configurations received identical parsed resume fields and identical job description text to isolate the effect of the matching method itself.

E. Stage 3: Adaptive Interview Generation and Evaluation

For candidates with $S_{\text{resume}} \geq \tau_{\text{ats}}$ an interview plan of up to M questions was generated conditioned on (R, J) . Each subsequent question was selected adaptively based on the candidate's prior answer, subject to a bound on the number of consecutive adaptive (i.e., non-plan) questions and a maximum total question count, consistent with the interview budget constraints of the underlying system (MAX_CONSECUTIVE_ADAPTIVE, MAX_PLAN_QUESTIONS). Each answer was scored by g_{ϕ} against a rubric **[REQUIRED SPECIFICATION: rubric dimensions — e.g., technical correctness, relevance, completeness — and whether each dimension is scored independently and summed/averaged, or scored holistically]** to produce a per-turn score, and turn scores were aggregated into $S_{\text{interview}}$. A guardrail layer rejected off-topic or non-responsive answers up to a bound (MAX_REJECTS_PER_TURN), after which the system forced acceptance of the answer as scored (forced_accept), to prevent an adversarial or evasive candidate from stalling the interview indefinitely.

F. Stage 4: Integrity and Proctoring Analysis

During the interview session, discrete integrity events (e.g., multiple faces detected, prolonged absence from frame, tab-switch events) were logged with an associated severity level $\in \{\text{critical}, \text{high}, \text{medium}, \text{low}\}$. The integrity score was computed as

$$c(E) = \max\left(0, 100 - \sum_{e \in E} w(e)\right)$$

where $w(e)$ denotes the severity weight of event e , and repeated events of the same kind were down-weighted (weight halved on repetition) to prevent a single persistent minor condition from dominating the score. A session containing at least one *critical* event was floored at a review verdict regardless of the numeric score. Consistent

with Section VII (Ethics), $c(E)$ and its associated events are reported to the recruiter as an *integrity signal for human review*, not as an automated determination of misconduct.

G. Algorithm

Algorithm 1 End-to-End Candidate Evaluation

Input: resume R , job description J , threshold τ_{ats} , max questions M
Output: ranked report entry (S_{resume} , $S_{\text{interview}}$, $c(E)$, S_{final})

```

1:  $R_{\text{parsed}} \leftarrow \text{Parse}(R)$ 
2:  $S_{\text{resume}} \leftarrow f_{\theta}(R_{\text{parsed}}, J)$ 
3: if  $S_{\text{resume}} < \tau_{\text{ats}}$  then
4:   return Reject( $S_{\text{resume}}$ )           ▷ no interview
stage entered
5: end if
6:  $\text{plan} \leftarrow \text{GeneratePlan}(R_{\text{parsed}}, J, M)$ 
7:  $E \leftarrow \emptyset$ ;  $\text{turn\_scores} \leftarrow []$ 
8: for  $t = 1$  to  $|\text{plan}|$  do
9:    $\text{question\_t} \leftarrow \text{SelectQuestion}(\text{plan}, \text{turn\_scores}, t)$  ▷ adaptive
10:   $\text{answer\_t}, \text{events\_t} \leftarrow \text{Administer}(\text{question\_t})$ 
11:   $E \leftarrow E \cup \text{events\_t}$ 
12:   $\text{score\_t} \leftarrow g_{\phi}(\text{answer\_t}, \text{question\_t}, J, R_{\text{parsed}})$ 
13:   $\text{turn\_scores.append}(\text{score\_t})$ 
14: end for
15:  $S_{\text{interview}} \leftarrow \text{Aggregate}(\text{turn\_scores})$ 
16:  $c(E) \leftarrow \text{IntegrityScore}(E)$ 
17:  $S_{\text{final}} \leftarrow h(S_{\text{resume}}, S_{\text{interview}}, c(E))$ 
18: return ( $S_{\text{resume}}$ ,  $S_{\text{interview}}$ ,  $c(E)$ ,  $S_{\text{final}}$ )

```

H. Dataset and Preprocessing

Two publicly available resume corpora were considered as candidate sources for the resume–job description matching experiments (Section III-D):

Dataset 1 — "Resume Dataset" (snehaanbhawal, Kaggle). This dataset contains 2,484 resumes across four fields — ID, Resume_str (plain-text resume), Resume_html, and Category — spanning 24 job categories (e.g., HR, Information-Technology, Engineering, Healthcare, Finance). Resumes were obtained by the original dataset authors via web scraping of publicly listed resume examples on livecareer.com. [REQUIRED VERIFICATION: license terms for redistribution/derivative use, since these are third-party-scraped example resumes rather than real applicant data; this must be confirmed against the dataset's Kaggle license field before publication.]

Dataset 2 — "Resume Dataset" (saugataroyarghya, Kaggle). [DATASET NOT INDEPENDENTLY VERIFIED: at the time of writing, the exact record count, field schema, and license for this dataset could not be confirmed through available search tools, and I was not able to load the dataset page directly. Before this dataset is

used or cited in the paper, it must be opened directly on Kaggle and its fields, size, and license documented in full — do not proceed to preprocessing or evaluation with this source until that verification step is complete.]

Preprocessing. Because neither corpus provides paired job descriptions or resume–job relevance labels, the following preprocessing decisions apply: (i) resumes were deduplicated by exact text match; (ii) resumes shorter than [REQUIRED THRESHOLD] tokens were excluded as likely parsing failures; (iii) Category labels were used only to construct category-consistent (resume, job description) pairs for matching experiments, using job descriptions drawn from the project's existing data/roles.json role definitions (Amazon/Deloitte roles) rather than from the Kaggle corpora, since the corpora do not themselves contain job descriptions. **This is a critical gap: neither dataset provides ground-truth resume–job relevance judgments**, so Experiment 1 (Section IV) requires either (a) a human annotation protocol producing 0–4 relevance ratings per (resume, job description) pair from 3–5 independent raters, as specified in Section IV-B, or (b) restriction of Datasets 1–2 to a category-matching proxy task (does the resume's labeled category match the job description's category?) with the explicit limitation that category match is a weak proxy for graded relevance and does not substitute for expert judgment. [REQUIRED DATA: human annotation protocol execution, or explicit adoption of the category-matching proxy with its limitation stated in Section VIII.]

I. Baselines, Metrics, and Experimental Setup

Baselines. The proposed matching method was compared against: (1) TF-IDF with cosine similarity; (2) BM25; (3) Sentence-BERT embedding cosine similarity; (4) zero-shot LLM matching (the same underlying LLM as the proposed method, prompted without the structured rubric used by the proposed method). Interview evaluation and the full pipeline were additionally compared against the ablation configurations defined in Section V (resume-only; resume + static interview; resume + adaptive interview; full pipeline).

Metrics. Ranking quality was measured using Precision@K, Recall@K, MRR, and NDCG@K over the human-annotated or category-proxy relevance labels described in Section III-H. Rank agreement between matching methods was measured using Spearman's ρ and Kendall's τ . Latency was measured as wall-clock time per candidate–job pair, averaged over [REQUIRED: N] runs, and reported with standard deviation. [REQUIRED SPECIFICATION: exact value of K used for Precision@K/Recall@K/NDCG@K, and justification for that choice.]

Experimental setup. [REQUIRED SPECIFICATION — hardware: CPU/GPU model, RAM, VRAM actually used to run these experiments; software: exact package versions for the embedding model (e.g., sentence-transformers

version and checkpoint name), the LLM provider and model identifier(s) used for the proposed method and the zero-shot baseline, temperature and token-limit settings, and random seed(s). None of these values are stated here because they have not yet been fixed and executed; inventing plausible-looking hardware or hyperparameter values would misrepresent a reproducibility section that reviewers rely on verbatim, so this subsection must be completed from the actual run configuration before submission, not filled with representative defaults.]

Two things worth flagging directly rather than glossing over:

1. **Dataset 2 is unverified.** My searches did not return the saugataroyarghya/resume-dataset page itself (Kaggle's dataset pages aren't reliably indexed by general search, and I don't have a Kaggle-specific tool available). I did not fabricate its row count or schema — I marked it as [DATASET NOT INDEPENDENTLY VERIFIED]. If you can paste the dataset's own "Data" tab content (fields, row count, license), I'll fold in verified numbers.
2. **Neither dataset has resume-job description pairs or relevance labels,** which is a structural problem for Experiment 1 as you've defined it — I flagged this as a required decision point (human annotation vs. category-proxy) rather than silently picking one for you, since it changes what the paper can honestly claim about ground truth.

RESULT AND CONCLUSION

1. Title options

Pick based on which contribution you can actually defend (see §4):

- "An Integrated Pipeline for AI-Assisted Resume Screening and Adaptive Interviewing: A System and Evaluation Framework" — honest if you evaluate the pipeline, not just build it
- "Can Adaptive Interviewing Correct Resume Screening Errors? A Controlled Audit of an AI Hiring Pipeline" — honest if Experiment 4 (inconsistency audit) is your centerpiece
- "Local versus Hosted LLMs in AI-Assisted Recruitment: A Quality-Latency-Privacy Trade-off Study" — honest only if Experiment 7 is run with real hardware measurements

Do not pick a title implying "novel AI hiring system" — the novelty claim you can defend is the *audit methodology*, not the pipeline itself (resume screening + LLM interviews are well-trodden).

2. Research gap

Based on what's actually distinctive here: most resume-screening literature evaluates screening in isolation

(Precision@K, NDCG against a static ground truth) and most interview-assessment literature evaluates interviews in isolation. [REQUIRED CITATION — verify this gap against recent SIGIR/RecSys/ACL work before asserting it; do not assert "no one has done this" without a lit search]. The defensible gap, *if the literature search confirms it*: few systems evaluate whether a **second-stage interaction (interview) can measurably correct ranking errors introduced by a first-stage static classifier (resume screening)**, using a controlled, labeled adversarial design (over-seller / hidden-gem) rather than post-hoc case studies.

3. Research questions (narrowed to what's actually testable with this codebase)

- **RQ1** (screening quality): How does the pipeline's resume-screening ranking compare to TF-IDF/BM25/SBERT/zero-shot-LLM baselines on Precision@K, NDCG@K?
- **RQ2** (correction): Does the adaptive interview stage recover hidden-gem candidates and demote over-sellers relative to resume-only ranking (ΔR , R_{rec})?
- **RQ3** (component attribution): Which pipeline stage contributes the improvement — static interview vs. adaptive vs. + structured pros/cons? (ablation)
- **RQ4** (robustness): What fraction of prompt-injection/adversarial resume inputs does the pipeline resist (IRR), and does resistance vary by attack type?
- **RQ5** (deployment trade-off): What is the quality/latency/resource cost of local (Ollama) vs. hosted (Gemini) deployment?
- Drop the human-agreement RQ (RQ2 in your original list) unless you can actually recruit 3+ independent evaluators — without them it's not answerable, and a paper claiming "AI-human agreement" with $n=1$ evaluator (you) will be rejected on sight.

4. Contributions — audited against what's implemented

Candidate contribution	Verdict
Integrated pipeline (screening+interview+integrity)	C — design decision, not itself a research contribution. State it as system description, not a claim.
Evaluation framework across the pipeline	Defensible only if you build and run Experiments 1–6 below. Currently: D (hypothesis)
Inconsistency-audit methodology (Case A/B)	Your strongest candidate — genuinely a methodological

	contribution if it produces real $\Delta R/R_{\text{rec}}$ numbers on a labeled synthetic set
Baseline comparison (BM25/SBERT/LLM)	Standard, defensible once run — not novel by itself, but necessary evidence
Local vs. hosted trade-off	Defensible with real measurements; useful but modest contribution (engineering benchmark, not a new method)
Robustness/IRR	Defensible as a narrow robustness claim against a <i>stated</i> attack set only — never "secure against prompt injection"
Fairness/bias auditing	Not a contribution. prd.md explicitly excludes this. Do not claim it.

Realistic framing for a workshop paper: contributions 2 (partial) + 3 + 4, with 5 and 6 as secondary evidence.

5. Experimental matrix

#	Experiment	Answers	Status
1	Screening vs. 4 baselines (P@K, NDCG, MRR)	RQ1	[REQUIRED EXPERIMENT]
2	Human agreement (screening)	dropped unless evaluators recruited	[REQUIRED HUMAN EVALUATION]
3	Human agreement (interview scoring)	dropped unless evaluators recruited	[REQUIRED HUMAN EVALUATION]
4	Inconsistency audit (Case A/B, ΔR , R_{rec})	RQ2	[REQUIRED EXPERIMENT] — highest priority
5	Ablation (A0–A5 from publications.md §6.6)	RQ3	[REQUIRED EXPERIMENT]

6	Adversarial/IRR	RQ4	[REQUIRED EXPERIMENT]
7	Local vs. hosted (latency/resource/quality)	RQ5	[REQUIRED EXPERIMENT] + [REQUIRED DATA: hardware spec]

6. Datasets

No public dataset gives you ground-truth resume $\leftrightarrow$ JD relevance for *this* specific pipeline. Options, honestly assessed:

- **Real public resume/JD datasets** (e.g., Kaggle resume-classification sets): usable for RQ1 baseline comparison only, not for the audit (no controlled over-seller/hidden-gem labels exist in the wild). [REQUIRED DATA: verify license and PII status before touching any of these — resumes are PII-heavy]
- **Synthetic resumes generated by your own generator**: appropriate *only* for the controlled audit (Exp. 4) — must be explicitly labeled synthetic in every table/figure, never presented as real hiring evidence, per your own standing rule.
- No dataset currently gives you human ground truth. That has to be built (below).

7. Ground-truth strategy

For Exp. 1 (screening quality): construct N resumes $\times$ M job descriptions and have 3 independent evaluators rate relevance 0–4 (rubric as you proposed), report Fleiss' κ . [REQUIRED HUMAN EVALUATION — you do not currently have evaluators; this is the actual blocker for RQ1's strongest form]. If you cannot recruit evaluators, RQ1 must be reframed as "agreement with lexical/embedding baselines" rather than "accuracy," and that limitation must be stated explicitly, not hidden.

For Exp. 4 (audit): ground truth is **constructed by design** (you assign competence vectors when generating synthetic candidates), not annotated — this is a real strength, but state plainly in Threats to Validity that this is construct validity, not real-hiring validity.

8. Baselines

TF-IDF+cosine, BM25, SBERT similarity, zero-shot LLM ranking, resume-only (no interview). All defensible, all cheap to implement, no excuse to skip any of them.

9. Metrics

Precision@K, NDCG@K, MRR, Kendall's τ (RQ1); ΔR , R_{rec} , Correction Ratio (RQ2); per-condition NDCG (RQ3, ablation); IRR (RQ4, explicitly scoped to the tested

attack set); latency (p50/p95), peak RAM/VRAM, tokens/sec (RQ5).

10. Ablations

A0 resume-only → A1 +static interview → A2 +adaptive interview → A3 +structured pros/cons → A4 full pipeline → A5 (optional) +integrity signal. Identical dataset and scoring procedure across all five — this is non-negotiable for the ablation to mean anything.

CONCLUSION

In essence, the described AI-powered recruitment solution combines the modern large language model features with a structured system design for producing highly accurate and context-dependent evaluations of applicants. Thus, the designed architecture manages to strike a good balance between intelligence and efficiency by making it possible to process resumes, analyze interview responses, and monitor candidate behavior in a single flow. The current approach stands out from the traditional solutions, which rely on keyword searching or manual processing of applications. Instead, by using LLM with context understanding capability, the proposed system provides deeper insights into candidate qualifications and experience and allows evaluating applicants objectively. To be specific, the backbone of the suggested system is the LLM, which has been installed locally on an Ollama server using the LLaMA 3.1 model. Resumes are processed using the PyMuPDF (fitz) library to extract necessary pieces of text, such as skills, experience, and educational background. The extracted data will be analyzed using the AI model without any dependence on key terms or individual factors. Indeed, just like a human recruiter would do when looking at the applicant's profile, the model would understand the interrelationships between the mentioned factors.

FUTURE SCOPE

The described system proves to be a highly promising solution in the field of AI recruitment but there is still a number of improvements that could potentially be made. First of all, an opportunity to implement learning within this framework should be considered since the model can improve with time when receiving feedback from hiring officers and recruiters. It may prove to be beneficial to employ either reinforcement learning or some other learning algorithms in this case. Secondly, natural language processing capabilities of this solution can be further enhanced with more sophisticated methods used by transformers and state-of-the-art models for fine-tuning and prompt engineering. The same goes for creating new datasets and implementing NER and semantic matching in order to increase parsing accuracy. Thirdly, there are a couple of improvements that can be implemented to further

develop the modules related to interviews and proctoring. They include adding support for more complicated tasks like speech recognition, sentiment analysis, and facial expressions recognition in order to evaluate not only the contents but also the way candidates communicate with the interviewer. As for the latter, more advanced anomaly detection techniques need to be employed. From the standpoint of system design, it may prove to be useful to replace JSON-based file structure and integrate some kind of database management into the design (e.g., PostgreSQL or MongoDB). This will make the solution scalable enough to handle large amounts of recruitment data. Moreover, it may be interesting to try implementing deployment to a cloud environment along with containers (Docker). Finally, it seems that it might prove useful to further evolve the created solution into the recruitment assistant that will automate various tasks from sending job descriptions to generating questions and communicating with candidates, thus fully automating the hiring process.

REFERENCES

- [1] A. Tiwari et al., "Applicant Tracking and Scoring System," *International Research Journal of Engineering and Technology (IRJET)*, vol. 6, no. 4, pp. 320–324, 2019.
- [2] C. Li, E. Fisher, R. Thomas, S. Pittard, V. Hertzberg, and J. D. Choi, "Competence-Level Prediction and Resume & Job Description Matching Using Context-Aware Transformer Models," in *Proc. 2020 Conf. Empirical Methods in Natural Language Processing (EMNLP)*, Online, Nov. 2020, pp. 8456–8466, doi: 10.18653/v1/2020.emnlp-main.679.
- [3] P. Pudasaini et al., "Job Recommendation System Using Word Embeddings and Gale–Shapley Stable Matching," 2022. [VENUE/DOI NOT INDEPENDENTLY CONFIRMED: this paper's existence and method were corroborated via a secondary citation in a CEUR-WS proceedings paper, but I could not directly load the original Springer chapter to verify its exact title, page numbers, and DOI as given in the source pool. Confirm against the publisher record before finalizing.]
- [5] C. Gan, Q. Zhang, and T. Mori, "Application of LLM Agents in Recruitment: A Novel Framework for Automated Resume Screening," *Journal of Information Processing*, vol. 32, pp. 881–893, 2024, doi: 10.2197/ipsjip.32.881.
- [6] Lo et al., "AI Hiring with LLMs: A Context-Aware and Explainable Multi-Agent Framework for Resume Screening," 2025. [VENUE NOT INDEPENDENTLY CONFIRMED: I located and used this paper's content (arXiv preprint) but could not confirm the CVPRW 2025 venue attribution given in the source pool. Verify the proceedings venue before finalizing.]

- [7] A. Fabris, N. Baranowska, M. J. Dennis, D. Graus, P. Hacker, J. Saldivar, F. Zuiderveen Borgesius, and A. J. Biega, "Fairness and Bias in Algorithmic Hiring: A Multidisciplinary Survey," *ACM Transactions on Intelligent Systems and Technology*, vol. 16, no. 1, Art. 16, pp. 1–54, 2025, doi: 10.1145/3696457.
- [8] P. Baxi, J. Xu, J. Y. Jiang, and S. Jasin, "Prompt Injection in Automated Résumé Screening with Large Language Models: Single and Multi-Injection Settings," in *Findings of the Association for Computational Linguistics: ACL 2026*, 2026.
- [9] M. Honnibal and I. Montani, "spaCy 2: Natural Language Understanding," 2017. [Online]. Available: <https://spacy.io/>
- [10] OpenAI, "GPT-4 Technical Report," 2023. [Online]. Available: <https://arxiv.org/abs/2303.08774>
- [11] J. Lee and S. Kim, "AI-Based Recruitment System Using Machine Learning," *IEEE Trans.*, 2021.
- [12] A. Fernandez et al., "Artificial Intelligence in Hiring: Trends and Applications," *IEEE Computer*, 2022.
- [13] Ollama, "Ollama: Run Large Language Models Locally," 2024. [Online]. Available: <https://ollama.com/>
- [14] S. Singh and P. Patel, "Online Proctoring System Using Computer Vision," in *IEEE Conf.*, 2021.
- [15] K. He et al., "Deep Residual Learning for Image Recognition," in *CVPR*, 2016.
- [16] PyMuPDF, "PyMuPDF Documentation (fitz)," 2023. [Online]. Available: <https://pymupdf.readthedocs.io/>
- [17] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.
- [18] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*. O'Reilly, 2009.
- [19] A. Krizhevsky et al., "ImageNet Classification with Deep Convolutional Neural Networks," in *NeurIPS*, 2012.
- [20] T. B. Brown et al., "Language Models are Few-Shot Learners," in *Proc. NeurIPS*, 2020.
- [21] A. Radford et al., "Whisper: Robust Speech Recognition via Large-Scale Weak Supervision," 2022.
- [22] M. Malinowski et al., "AI in Human Resource Management: A Review," *IEEE Access*, 2021.
- [23] Streamlit Inc., "Streamlit Documentation," 2023. [Online]. Available: <https://docs.streamlit.io/>
- [24] Python Software Foundation, "Python Documentation," 2023. [Online]. Available: <https://docs.python.org/3/>
- [25] G. Dutta, "Resume Dataset," *Kaggle*, 2021. [Online]. Available: <https://www.kaggle.com/datasets/gauravduttakiit/resume-dataset>
- [1] K. Gweon et al., "Automated Resume Screening Using Natural Language Processing," *IEEE Access*, 2020.
- [2] M. Raghavan et al., "Mitigating Bias in Algorithmic Hiring," in *Proc. FAccT*, 2020.
- [3] A. Vaswani et al., "Attention Is All You Need," in *Advances in Neural Information Processing Systems*, 2017.
- [4] J. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL*, 2019.
- [5] H. Touvron et al., "LLaMA: Open and Efficient Foundation Language Models," 2023. [Online]. Available: <https://arxiv.org/abs/2302.13971>
- [6] H. Touvron et al., "LLaMA 2: Open Foundation and Fine-Tuned Chat Models," 2023. [Online]. Available: <https://arxiv.org/abs/2307.09288>
- [7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in *Proc. EMNLP*, 2019.